

Module 2.1: NoSQL Databased Models

Contents

NoSQL Databases	1
Need for NoSQL Databases	2
Characteristics of NoSQL Databases	2
Advantages of NoSQL Databases	2
Aggregate Data Models	3
Importance of Aggregate Data Models	3
Benefits of Aggregate-Oriented Design	3
Aggregates	4
Aggregate Boundaries	4
Uses of Aggregates	4
Advantages of Aggregates	5
Key-Value Data Model	5
Document Data Model	6
Relationships in NoSQL Databases	7
Graph Databases	8
Nodes	8
Edges	8
Properties	9
Schemeless Databases	10
Materialized Views	11
Working of Materialized Views	11

NoSQL Databases

NoSQL (Not Only SQL) refers to a category of database management systems that store and manage data in formats other than traditional relational tables. NoSQL databases are designed to handle large volumes of structured, semi-structured, and unstructured data while providing high scalability and performance. **Key Points**

- NoSQL stands for "**Not Only SQL.**"
- Uses flexible data models instead of fixed table structures.
- Supports distributed storage across multiple servers.
- Suitable for Big Data and real-time web applications.
- Provides horizontal scalability.
- Redis
- HBase
- CouchDB

Examples

- MongoDB
- Cassandra

Applications Requiring NoSQL

- Social media platforms
- E-commerce websites
- IoT systems
- Cloud applications
- Real-time analytics systems

Need for NoSQL Databases

Traditional relational databases face challenges when dealing with massive data volumes, high-speed transactions, and rapidly changing data structures. NoSQL databases were developed to overcome these limitations. **Reasons for the Need of NoSQL**

1. Handling Big Data

- Modern applications generate huge amounts of data.
- NoSQL databases efficiently store and process large datasets.

2. High Scalability

- Data can be distributed across multiple servers.
- Supports horizontal scaling by adding new machines.

3. Flexible Data Structure

- No fixed schema is required.

- Easy to modify data formats without redesigning tables.

4. High Performance

- Provides faster read and write operations.
- Suitable for real-time applications.

5. Distributed Environment Support

- Designed for cloud and distributed computing systems.
- Ensures availability and fault tolerance.

Characteristics of NoSQL Databases

1. Schema-less Design

- Data structure can vary from one record to another.
- No need to define tables before storing data.

2. Horizontal Scalability

- Capacity increases by adding more servers.
- Supports large-scale distributed systems.

3. High Availability

- Data is replicated across multiple nodes.
- System remains operational even if a server fails.

4. Distributed Architecture

- Data is stored on multiple machines.
- Improves reliability and performance.

5. Support for Multiple Data Models

- Key-Value Model
- Document Model
- Column-Family Model
- Graph Model

6. High Performance

- Optimized for fast data access and processing.
- Suitable for real-time applications.

7. Fault Tolerance

- Continues operation despite hardware failures.
- Uses replication and data redundancy.

Advantages of NoSQL Databases

1. Scalability

- Easily scales by adding more servers.
- Handles growing data volumes efficiently.

2. Flexibility

- Supports dynamic and changing data structures.
- No fixed schema restrictions.

3. High Speed

- Faster data insertion and retrieval.
- Suitable for real-time applications.

4. Cost-Effective

- Can run on inexpensive commodity hardware.
- Reduces infrastructure costs.

5. High Availability

- Replication ensures continuous service.
- Minimizes downtime.

6. Better Handling of Big Data

- Designed to manage massive datasets.
- Supports distributed data storage.

7. Supports Diverse Data Types

- Structured data
- Semi-structured data
- Unstructured data

Aggregate Data Models

An **Aggregate Data Model** is a data modeling approach in which related data items are grouped together and treated as a single unit called an **aggregate**. Instead of storing data in multiple related tables, aggregate-oriented databases store associated information together, making data retrieval faster and simpler. **Key Points**

- An aggregate is a collection of related data treated as one unit.
- Commonly used in NoSQL databases.
- Reduces the need for complex joins.
- Improves performance and scalability.
- Suitable for distributed database systems.

Here, all order-related information is stored together as a single aggregate.

Example

Customer Order Aggregate:

```
Order
├── Order ID: 1001
├── Customer Name: John
├── Products:
│   ├── Laptop
│   └── Mouse
└── Total Amount: $1200
```

Importance of Aggregate Data Models

Aggregate data models play a crucial role in NoSQL databases by simplifying data management and improving system performance.

1. Efficient Data Access

- Related data is stored together.
- Fewer database operations are required.

2. Improved Performance

- Eliminates expensive join operations.
- Faster query execution.

3. Better Scalability

- Aggregates can be distributed across multiple servers.

- Supports horizontal scaling.

4. Simplified Data Management

- Easier to maintain and update related information.
- Reduces data complexity.

5. Supports Distributed Systems

- Aggregates become natural units for partitioning and replication.

Benefits of Aggregate-Oriented Design

Aggregate-oriented design focuses on organizing data into aggregates to optimize storage and access.

1. Reduced Database Joins

- Related information is stored together.
- Minimizes join operations.

2. Faster Read Operations

- Entire aggregate can be retrieved in a single query.

- Improves response time.

3. Easier Data Distribution

- Aggregates can be partitioned across servers.
- Supports distributed architectures.

4. Better Consistency Management

- Updates occur within a single aggregate.
- Reduces synchronization issues.

5. Improved Scalability

- Aggregates can be independently scaled.

- Handles growing workloads efficiently.

6. Simplified Development

- Data structure closely matches application objects.
- Easier for developers to work with.

Aggregates

An **Aggregate** is a collection of related data elements that are treated as a single unit for storage, retrieval, and updates in a database. In NoSQL databases, aggregates are often stored as a single document, record, or key-value structure. **Key Points**

- Basic building block of aggregate-oriented databases.
- Groups related data together.
- Acts as a single unit for operations.
- Improves performance and scalability.

All customer-related information forms one aggregate.

Example

Customer Aggregate:

```
Customer
├── Customer ID
├── Name
├── Address
├── Phone Number
└── Orders
```

Aggregate Boundaries

Aggregate boundaries define which data belongs inside an aggregate and which data should remain outside it. **Purpose of Aggregate Boundaries**

- Control data organization.
- Improve performance.
- Maintain consistency.
- Simplify updates.

Guidelines for Defining Boundaries

1. **Keep Closely Related Data Together:** Frequently accessed data should be within the same aggregate.
2. **Limit Aggregate Size:** Very large aggregates can reduce performance.
3. **Maintain Transaction Consistency:** Data requiring atomic updates should belong to the same aggregate.
4. **Separate Independent Data:** Unrelated information should be stored in different aggregates.

Order and Customer Profile may be stored as separate aggregates.

Example

```
Order Aggregate
├── Order Details
├── Product Details
└── Payment Information

Customer Profile
├── Customer Name
├── Address
└── Contact Information
```

Uses of Aggregates

Aggregates are widely used in NoSQL database systems for efficient data handling.

1. **Data Storage:** Stores related data as a single unit.
2. **Data Retrieval:** Retrieves complete information with one query.

3. **Data Updates:** Updates multiple related fields together.

4. **Data Partitioning:** Aggregates can be distributed across servers.

5. **Replication:** Replicated as complete units in distributed systems.

Common Applications

- E-commerce orders
- Customer profiles
- Banking transactions
- Social media posts
- Inventory management systems

Advantages of Aggregates

1. **Faster Queries** – Related data is stored together, reducing retrieval time.
2. **Improved Scalability** – Aggregates can be easily distributed across multiple servers.
3. **Better Performance** – Fewer joins and database operations improve efficiency.
4. **Simplified Transactions** – All related updates can be performed within a single aggregate.
5. **Easier Maintenance** – Organized data structures make management and updates simpler.
6. **Efficient Replication** – Complete aggregates can be replicated easily across database nodes.

Key-Value Data Model

The **Key-Value Data Model** is a NoSQL database model that stores data as a collection of **key-value pairs**, where each key is unique and is used to access its corresponding value. **Key Points**

- Simplest NoSQL data model.
- Each key uniquely identifies a value.
- Values can be simple data or complex objects.
- Provides fast data retrieval.
- Suitable for distributed systems.

Here, **101, 102, and 103** are unique keys used to retrieve their respective values.

Example

Key	Value
101	John
102	Alice
103	David

Popular Key-Value Databases

- Redis
- Amazon DynamoDB
- Voldemor
- Riak
- Berkeley DB

Structure of Key-Value Databases

A key-value database consists of two basic components:

1. Key

- Unique identifier.
- Used to locate data quickly.
- Cannot be duplicated.

2. Value

- Actual data associated with the key.
- Can be text, numbers, images, documents, or objects.

Structure

Key → Value

Example

```
Student_ID → {  
    Name : "Rahul",  
    Age : 20,  
    Course : "BCA"  
}
```

Working Process

1. Data is stored as key-value pairs.
2. User provides a key.
3. Database searches for the key.
4. Corresponding value is returned.
1. **Simple Structure** – Stores data as key-value pairs.
2. **Fast Data Access** – Retrieves data directly using unique keys.
3. **High Scalability** – Easily expands across multiple servers.

Features

4. **Flexible Data Storage** – Supports different types of data as values.
5. **Schema-less Design** – Does not require a predefined database schema.
6. **High Availability** – Ensures continuous operation through replication and fault tolerance.
7. **Efficient Performance** – Delivers high-speed processing for large-scale and real-time applications.

Advantages

1. **High Speed** – Direct key lookup enables rapid data access.
2. **Easy Scalability** – Additional servers can be added easily.
3. **Simplicity** – Easy to design, implement, and maintain.
4. **Flexibility** – Supports various data formats without restrictions.

5. **Cost-Effective** – Runs efficiently on low-cost hardware.
6. **High Availability** – Replication reduces downtime and data loss.

Applications

1. **Caching Systems** – Store frequently accessed data for faster retrieval.
2. **Session Management** – Maintain user session information in web applications.
3. **User Profiles** – Store and retrieve user information quickly.
4. **Shopping Carts** – Manage temporary customer purchase data.
5. **Gaming Applications** – Store player profiles, scores, and game states.
6. **Real-Time Analytics** – Process and analyze high-speed data efficiently.

Document Data Model

The **Document Data Model** is a NoSQL data model that stores data in the form of documents, typically using JSON, BSON, or XML formats.

- Data is stored as self-contained documents.
- Each document contains related information together.
- Documents can have different structures.
- No fixed schema is required.
- Commonly used in NoSQL databases like MongoDB.

Document Structure

1. A document consists of field-value pairs.
2. Fields represent attributes of the data.
3. Values can be text, numbers, arrays, or nested documents.
4. Each document has a unique identifier.
5. Documents are usually stored in collections.

```
{
  "StudentID": 101,
  "Name": "Rahul",
  "Age": 20,
  "Course": "BCA"
}
```

Features

1. **Schema-less Design** – Documents can have different structures.
2. **Flexible Storage** – Supports structured and semi-structured data.
3. **Nested Data Support** – Documents can contain sub-documents and arrays.
4. **High Scalability** – Easily distributed across multiple servers.
5. **Fast Retrieval** – Related data is stored within a single document.
6. **Self-Describing Data** – Documents include both data and structure.
7. **Easy Modification** – New fields can be added without redesigning the database.

Advantages

1. **Flexibility** – Supports changing data requirements without schema modifications.
2. **High Performance** – Reduces the need for complex joins.
3. **Scalability** – Supports horizontal scaling across servers.
4. **Easy Development** – Closely matches application objects and data structures.
5. **Efficient Storage** – Stores related data in a single document.
6. **Quick Access** – Retrieves complete records with fewer database operations.
7. **Cost-Effective** – Works efficiently on distributed commodity hardware.

Applications

1. **Content Management Systems** – Store articles, blogs, and media content.
2. **E-Commerce Platforms** – Manage product catalogs and customer information.
3. **User Profile Management** – Store user details and preferences.
4. **Mobile Applications** – Handle dynamic and rapidly changing data.
5. **Social Media Platforms** – Store posts, comments, and user interactions.
6. **Real-Time Analytics** – Process large volumes of semi-structured data.
7. **Catalog Management Systems** – Maintain product and inventory information.

Relationships in NoSQL Databases

A **relationship** in a NoSQL database represents the association between two or more data entities.

- Relationships connect related pieces of data.
- They show how one entity is linked to another.
- NoSQL handles relationships differently from relational databases.
- Relationships can be embedded or referenced.
- Proper relationship design improves data access efficiency.

A. Embedded Relationships: An **embedded relationship** stores related data inside the same document.

- Related data is stored within a parent document.
- Data can be retrieved in a single database query.
- Reduces the need for joins and lookups.
- Suitable for closely related data.
- Improves read performance.

```
{
  "CustomerID": 101,
  "Name": "Rahul",
  "Orders": [
    {"OrderID": 1, "Amount": 500},
    {"OrderID": 2, "Amount": 800}
  ]
}
```

Here, order details are embedded within the customer document.

B. Reference Relationships: A **reference relationship** stores related data in separate documents and links them using identifiers.

- Related data is stored in different documents.
- Documents are connected using reference IDs.
- Reduces data duplication.
- Suitable for large or frequently changing data.
- Requires additional queries to retrieve related data.

Here, CustomerID acts as a reference between the documents.

```
Customer
{
  "CustomerID": 101,
  "Name": "Rahul"
}

Order
{
  "OrderID": 1,
  "CustomerID": 101,
  "Amount": 500
}
```

Relationship Handling in NoSQL

1. NoSQL databases avoid complex joins used in relational databases.
2. Embedded relationships are used for tightly connected data.
3. Reference relationships are used for independent or large datasets.
4. Relationship handling depends on application requirements.
5. Aggregates help manage related data efficiently.
6. Denormalization is often used to improve performance.
7. Different NoSQL databases use different approaches to manage relationships.

Graph Databases

A Graph Database is a type of NoSQL database that stores data in the form of a graph. Instead of organizing data into tables and rows like relational databases, graph databases use nodes, edges, and properties to represent and store information. They are specifically designed to handle highly connected data and make relationship-based queries faster and more efficient.

Graph databases are widely used in applications where relationships between data elements are as important as the data itself, such as social networks, recommendation systems, and fraud detection systems.

Nodes

Nodes are the fundamental units of a graph database. They represent entities, objects, or records within the database. Each node contains information about a specific item.

For example:

- A person in a social network.
- A product in an e-commerce website.
- A city in a transportation network.

```
Node: Student
Name = Rahul
Roll No = 101
Department = CSE
```

Each node can store multiple attributes or properties that describe the entity it represents.

Edges

Edges, also known as relationships, connect two nodes and describe how they are related. Unlike relational databases where relationships are established through foreign keys, graph databases store relationships directly as edges.

Examples:

- Student "ENROLLED IN" Course

- Customer "PURCHASED" Product
- User "FOLLOWS" User

Since relationships are stored explicitly, graph databases can traverse connected data much faster than traditional databases.

Example: Rahul ----ENROLLED_IN----> Database Course

Properties

Properties are key-value pairs used to store additional information about nodes and edges. They provide details and characteristics of entities and relationships.

Examples of node properties:	Examples of edge properties:
<pre> Student ----- Name : Rahul Age : 21 Department : CSE </pre>	<pre> ENROLLED_IN ----- Semester : 5 Grade : A </pre>

Features

- Stores data as nodes, edges, and properties.
- Efficiently handles highly connected data.
- Supports flexible and dynamic schema design.
- Provides fast relationship traversal and querying.
- Eliminates the need for complex table joins.
- Offers high performance for network-based applications.
- Supports real-time analysis of relationships.

Advantages

- Faster execution of relationship-based queries.
- Better performance for connected data.
- Flexible structure allows easy modification.

- Simplifies representation of complex networks.
- Reduces query complexity by avoiding joins.
- Supports scalability for large datasets.
- Improves visualization of interconnected information.

Applications

- **Social Networks:** Models user relationships and interactions.
- **Recommendation Systems:** Provides personalized recommendations.
- **Fraud Detection:** Detects suspicious connections and activities.
- **Network Management:** Manages communication and computer networks.
- **Knowledge Graphs:** Stores interconnected knowledge.
- **Route Planning:** Finds optimal travel routes.

Schemeless Databases

A Schemeless Database is a type of database that does not require a predefined schema or fixed structure for storing data. Unlike traditional relational databases, where tables and columns must be defined before data insertion, schemeless databases allow records to have different structures within the same collection.

These databases are commonly associated with NoSQL systems and are designed to handle unstructured and semi-structured data efficiently. The flexible nature of schemeless databases makes them suitable for modern applications where data requirements change frequently.

Example:

Both documents can exist in the same collection even though they contain different fields.

Document 1:

```
{
  "Name": "Rahul",
  "Age": 21,
  "Department": "CSE"
}
```

Document 2:

```
{
  "Name": "Priya",
  "Department": "IT",
  "City": "Kolkata"
}
```

Characteristics

- **Flexible Schema:** No fixed structure is required.
- **Dynamic Data Model:** Records can have different fields.
- **Unstructured Data Support:** Stores diverse data formats.
- **Easy Scalability:** Supports distributed storage.
- **Rapid Development:** Simplifies application development.
- **High Availability:** Provides fault tolerance and reliability.

Challenges

- **Data Inconsistency:** Difficult to maintain uniformity.
- **Weak Validation:** Allows invalid data entries.
- **Complex Management:** Harder to organize data.
- **Query Issues:** Can affect query efficiency.
- **Data Redundancy:** May create duplicate data.
- **Governance Concerns:** Requires extra control measures.

Advantages

- **Flexibility:** Easily adapts to changes.
- **Faster Development:** Reduces design effort.
- **Diverse Data Handling:** Supports multiple data types.
- **Scalability:** Handles growing data volumes.
- **Reduced Maintenance:** Minimizes schema updates.
- **Agile Support:** Fits rapidly changing environments.

Applications

- **CMS:** Stores content and media files.
- **Social Media:** Manages user-generated content.
- **E-Commerce:** Stores diverse product data.
- **Big Data:** Handles massive datasets.
- **IoT:** Stores sensor-generated data.
- **Web and Mobile Apps:** Supports dynamic applications.
- **Real-Time Analytics:** Processes live data streams.

Materialized Views

A Materialized View is a database object that stores the result of a query physically in the database. Unlike a normal view, which executes the underlying query each time it is accessed, a materialized view stores the query results and retrieves them directly when requested.

Materialized views are primarily used to improve query performance, especially when dealing with large datasets, complex joins, aggregations, and frequently executed queries.

For example, if a company frequently generates a report showing total sales by region, the result can be stored in a materialized view instead of recalculating it every time the report is requested.

Working of Materialized Views

A materialized view is created based on a query involving one or more tables. When the view is created, the query is executed and its result is stored physically on disk.

Whenever users access the materialized view, the database retrieves the stored data directly rather than executing the original query again. This significantly reduces query execution time.

However, when the data in the underlying tables changes, the materialized view may become outdated. Therefore, it must be refreshed periodically to synchronize its contents with the source tables.

Steps in Working

1. Data is stored in one or more base tables.
2. A query is executed to create a materialized view.
3. The query result is physically stored in the database.
4. Users access the stored results directly.
5. The materialized view is refreshed when underlying data changes.

Example: Instead of calculating totals repeatedly, the database stores the result in the materialized view.

Sales Table	

Region	Sales
East	50000
West	40000
North	35000
Materialized View	

Region-wise Total Sales	

Benefits

- **Improved Performance:** Speeds up query execution.
- **Reduced Cost:** Lowers processing overhead.
- **Faster Reporting:** Generates reports quickly.
- **Large Database Support:** Improves system efficiency.
- **Data Warehousing:** Enhances analytical processing.
- **Reduced Traffic:** Minimizes data transfer.

Applications

- **Data Warehousing:** Stores summarized data.
- **Business Intelligence:** Supports dashboards and reports.
- **Decision Support:** Aids business decisions.
- **Financial Reporting:** Generates financial summaries.
- **E-Commerce Analytics:** Analyzes sales and customers.
- **Performance Optimization:** Speeds up frequent queries.